Unification Algorithm

function UNIFY(x, y, θ) **returns** a substitution to make x and y identical
  inputs: x, a variable, constant, list, or compound
    y, a variable, constant, list, or compound
    θ, the substitution built up so far

  if θ = failure **then return** failure
  else if $x = y$ **then return** θ
  else if VARIABLE?(x) **then return** UNIFY-VAR(x, y, θ)
  else if VARIABLE?(y) **then return** UNIFY-VAR(y, x, θ)
  else if COMPOUND?(x) **and** COMPOUND?(y) **then**
    return UNIFY(ARGS[x], ARGS[y], UNIFY(OP[x], OP[y], θ))
  else if LIST?(x) **and** LIST?(y) **then**
    return UNIFY(REST[x], REST[y], UNIFY(FIRST[x], FIRST[y], θ))
  else return failure

Unification Algorithm: Handling Variables

function $\text{Unify-Var}(var, x, \theta)$ **returns** a substitution
inputs: var, a variable
x, any expression
θ, the substitution built up so far

if $\{var/val\} \in \theta$ **then return** $\text{Unify}(val, x, \theta)$
else if $\{x/val\} \in \theta$ **then return** $\text{Unify}(var, val, \theta)$
else if $\text{Occur-Check?}(var, x)$ **then return** failure
else return add $\{var/x\}$ to θ

Comprehensive Example

Story in plain English:
Luna is a cat.
Either Jack or Curiosity Killed Luna.
Nothing is killed by both Curiosity and Jack.
Animal lovers do not kill animals.
Everyone who owns a dog loves animals.
Every person owns a dog.
(Missing: commonsense knowledge! Added below.)
Jack is a person.
All cats are animals.

Write as FOL KB

Constants list: Jack, Luna, Cur

Predicates list: Cat(x), K(x,y), AL(x), An(x), Dog(x), Own(x,y), P(x)

Luna is a cat.	Cat(Luna)
Either Jack or Curiosity Killed Luna.	K(Jack,Luna) v K(Cur,Luna)
Nothing is killed by both Curiosity and Jack.	forall x ~(K(Jack,x) ^ K(Cur,x))
Animal lovers do not kill animals.	forall x AL(x) => ~(exists y An(y) ^ K(x,y))
Everyone who owns a dog loves animals.	forall x (exists y Dog(y) ^ Own(x,y)) => AL(x)
Every person owns a dog.	forall x P(x) => (exists y Dog(y) ^ Own(x,y))
Jack is a person.	P(Jack)
All cats are animals.	forall x Cat(x) => An(x)

Convert to CNF

1. Cat(Luna) ; Already a CNF clause (number the CNF clauses for future use)

2. K(Jack,Luna) v K(Cur,Luna) ; Already a CNF clause

forall x ~(K(Jack,x) ^ K(Cur,x)) ; Need to move negation inwards

forall x ~K(Jack,x) v ~K(Cur,x) ; Can now remove quantifiers, and we have a CNF clause

3. ~K(Jack,x) v ~K(Cur,x))

forall x AL(x) => ~(exists y An(y) ^ K(x,y)) ; Need to get rid of implication

forall x ~AL(x) v ~(exists y An(y) ^ K(x,y)) ; Need to move negation inwards

forall x ~AL(x) v forall y ~(An(y) ^ K(x,y)) ; one more negation move needed

forall x ~AL(x) v forall y ~An(y) v ~K(x,y) ; Can now remove quantifiers, and we have a CNF clause

4. ~AL(x) v ~An(y) v ~K(x,y)

Convert to CNF

forall x (exists y Dog(y) ^ Own(x,y)) => AL(x) ; Need to get rid of implication

forall x ~(exists y Dog(y) ^ Own(x,y)) v AL(x)   ; Need to move negation inwards

forall x forall y ~(Dog(y) ^ Own(x,y)) v AL(x)   ; Still need to move negation

forall x forall y ~Dog(y) v ~Own(x,y) v AL(x)   ; Can now remove quantifiers to get CNF clause

5. ~Dog(y) v ~Own(x,y) v AL(x)

forall x P(x) => (exists y Dog(y) ^ Own(x,y))   ; Need to get rid of implication

forall x ~P(x) v (exists y Dog(y) ^ Own(x,y))   ; Existential variable y replaced by Skolem function

forall x ~P(x) v (Dog(D12(x)) ^ Own(x,D12(x)))   ; Now can remove universal quantifier

~P(x) v (Dog(D12(x)) ^ Own(x,D12(x)))   ; Apply distributive rule, and get 2 CNF clauses

6.1 ~P(x) v Dog(D12(x))

6.2 ~P(x) v Own(x,D12(x)))

Convert to CNF

7. P(Jack) ; Already a CNF clause

forall x Cat(x) => An(x) ; Need to remove implication

forall x ~Cat(x) v An(x) ; Drop universal quantifier to get CNF clause

8. ~Cat(x) v An(x)

Resolution Proof

Prove: Curiosity killed the cat (Luna).

Q: K(Cur,Luna)

Negate Q, convert to CNF

Q': ~K(Cur,Luna) ; Already a CNF clause

Add Q' temporarily to KB, and derive a contradiction (empty clause) to prove Q

Resolution Proof

KB was:

1. Cat(Luna)
2. K(Jack,Luna) v K(Cur,Luna)
3. ~K(Jack,x) v ~K(Cur,x))
4. ~AL(x) v ~An(y) v ~K(x,y)
5. ~Dog(y) v ~Own(x,y) v AL(x)

6.1 ~P(x) v Dog(D12(x))

6.2 ~P(x) v Own(x,D12(x)))

7. P(Jack)
8. ~Cat(x) v An(x)

Negated query:

Q': ~K(Cur,Luna)

Resolve Q', 2, unifier { }

9. K(Jack,Luna}

Resolve 1, 8, unifier {x/Luna}

10. An(Luna)

Resolve 7, 6.1, unifier {x/Jack}

11. Dog(D12(Jack))

Resolve 7, 6.1, unifier {x/Jack}

12. Own(Jack,D12(Jack))

Resolve 11, 5, unifier {y/D12(Jack)}

13. ~Own(x, D12(Jack)) v AL(x)

Resolve 12, 13, unifier {x/Jack}

14. AL(Jack)

Resolve 14, 4, unifier {x/Jack}

15. ~An(y) v ~K(Jack,y)

Resolve 10, 16, unifier {y/Luna}

16. "K(Jack,Luna)

Resolve 9, 16, unifier { }

17. <empty clause>

Therefore Q is entailed by the KB

Resolution Proof: Comments

KB was:

1. Cat(Luna)

2. K(Jack,Luna) v K(Cur,Luna)

3. ~K(Jack,x) v ~K(Cur,x))

4. ~AL(x) v ~An(y) v ~K(x,y)

5. ~Dog(y) v ~Own(x,y) v AL(x)

6.1 ~P(x) v Dog(D12(x))

6.2 ~P(x) v Own(x,D12(x)))

7. P(Jack)

8. ~Cat(x) v An(x)

Clause 3 never used, not needed for proof.
Can resolve 2,3, unifier {x/Luna} with 2 possible results, eliminating EITHER first or second atoms
(NOT ALLOWED to eliminate both atoms!)
a. K(Jack,Luna)v~K(Jack,Luna)
b. K(Cur,Luna)v~K(Cur,Luna)
Both are OK and are added - useless tautologies.

Query might have been: “somebody killed Luna”:
Q1: exists x K(x,Luna)
Negating it we have:
not exists x K(x,Luna)
Converting this to CNF we get:
Q1’: ~K(x,Luna)
Then we can resolve Q1’, 2, unifier {x/Cur} to get:
9’: K(Jack,Luna)
and now continue the proof exactly as before.
The unifier {x/Cur} in the proof allows reconstruction of the fact that it is Cur that killed Luna.

Resolution Proof: Comments

KB was:

1. Cat(Luna)
2. K(Jack,Luna) v K(Cur,Luna)
3. ~K(Jack,x) v ~K(Cur,x))
4. ~AL(x) v ~An(y) v ~K(x,y)
5. ~Dog(y) v ~Own(x,y) v AL(x)

6.1 ~P(x) v Dog(D12(x))

6.2 ~P(x) v Own(x,D12(x)))

7. P(Jack)
8. ~Cat(x) v An(x)

Query might have been: "everything killed Luna":
Q2: forall x K(x,Luna)
Negating it we have:
not forall x K(x,Luna)
Converting to CNF we get (K123 is a Skolem constant)
Q2': ~K(K123,Luna)
This cannot be resolved with anything, and as the KB does not contain a contradiction we will be unable to prove Q2, which indeed is not entailed by the KB.